



Support Readiness Document Java™ 2 Standard Edition, Version 1.4 JavaBeans™ Component Architecture

Sun Microsystems, Inc.
901 San Antonio Road
Palo Alto, CA 94303
U.S.A. 650-960-1300

April 2002

© 2002 by Sun Microsystems, Inc.—Printed in USA.
901 San Antonio Road, Palo Alto, CA 94303-4900

All rights reserved. No part of this work covered by copyright may be duplicated by any means—graphic, electronic or mechanical, including photocopying, or storage in an information retrieval system—without prior written permission of the copyright owner.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.227-7013 (October 1988) and FAR 52.227-19 (June 1987). The product described in this manual may be protected by one or more U.S. patents, foreign patents, and/or pending applications.

TRADEMARKS: Java, Java 2 Platform, J2SE, Java 2 Enterprise Edition, J2EE, Java 2 SDK, JDK, Java Compatibility Kit, Java HotSpot, JavaBeans, Java Virtual Machine, JVM, Java Remote Method Invocation, Solaris, and Sun are trademarks or registered trademarks of Sun Microsystems, Inc. in the United States and other countries. UNIX is a registered trademark in the United States and other countries, exclusively licensed through X/Open Company, Ltd. Netscape is a trademark or registered trademark of Netscape Communications Corporation.

Table of Contents

1. JavaBeans™ Component Architecture Overview 1

- 1.1 Overview of the JavaBeans Component Model 1
 - 1.1.1 Services Provided by the JavaBeans Component Model 2
 - 1.1.2 Properties 3
 - 1.1.3 Event Registration 4
 - 1.1.4 Persistent Storage 4
 - 1.1.5 Introspection 5
 - 1.1.6 An Example of a Simple Bean 6
- 1.2 LTP for JavaBeans Overview 9
 - 1.2.1 JavaBeans and LTP 10
 - 1.2.2 Features, Advantages, and Benefits of LTP 10
 - 1.2.3 Features or Services Not Provided With LTP 11
- 1.3 Other Introductory Material 11
- 1.4 Specialized Terminology 11
 - 1.4.1 Glossary 11

2. Product Changes for Version 1.4 13

- 2.1 Previous Versions of LTP for JavaBeans 13
 - 2.1.1 Static Constants 13
 - 2.1.2 Array Elements 14
- 2.2 Backward and Forward Compatibility With Other Versions 14

3.	Using and Supporting LTP for JavaBeans	15
3.1	User's Guides and Tutorials	15
3.2	Using LTP for JavaBeans in a Development Environment	15
3.3	Tools and Utilities	16
3.4	Localization and Internationalization	16
4.	Code Samples	17
4.1	Writing an Object Graph to an <code>OutputStream</code>	17
4.2	Reading an Object Graph from an <code>InputStream</code>	18
4.3	Preventing Some Properties From Archiving	19
4.4	Using a <code>ByteArrayOutputStream</code>	21
5.	Troubleshooting	22
5.1	Missing No-Argument Constructor	22
5.2	Product Limitations	23
5.2.1	Anonymous Inner Classes are not Archivable	23
5.2.1.1	First Workaround: Make the Listener a Bean	24
5.2.1.2	Second Workaround: Write a Dynamic Proxy	24
5.3	Common User Questions	25
5.4	Troubleshooting Utilities	25
5.5	Common Developer Problems	26
5.5.1	Why Fields That Are Marked With the <code>Transient</code> Keyword Still Persist	26
5.6	Known Bugs and Their Workarounds	26
5.6.1	Bug 4646747: Custom persistence delegate lost after memory stress	27
6.	Key Files and Directories	28
6.1	Configuration Files	28
7.	Installing and Configuring LTP for JavaBeans	29
8.	Reference Information	30

8.1	Product Information	30
8.2	Technical Documentation	30
8.3	Frequently Asked Questions	30
8.4	Tutorials and Other References	31

Preface

This document provides support readiness information for the JavaBeans™ Component Architecture, which is part of the Java 2 Platform (J2SE™), Standard Edition, version 1.4.

This document is not designed to provide comprehensive product training. Instead, it focuses on issues immediately relevant to support, such as changes in this version of the product, using and troubleshooting the product, and installing and configuring the product. For pointers to other documentation, see [Section 8 “Reference Information.”](#)

1 JavaBeans™ Component Architecture Overview

The following SRD may contain actual software programs in source code form. This source code is made available for developers to use as needed, pursuant to the terms and conditions of this license located at:

<http://access1.sun.com/SRDs/BSD.html>

Two major features were added to the JavaBeans component architecture in J2SE, v. 1.4:

- Long Term Persistence for JavaBeans (LTP)

This is a text-based format for saving the state of JavaBeans components.

- `getFooListeners` JavaBeans specification amendment

This specifies that a `getFooListeners` method should be provided but is not strictly required for all listener types.

This document provides an overview of the JavaBeans component model as it exists in version 1.4. Because the `getFooListeners` change is straightforward, this document describes it only briefly at the end of [Section 1.1.3 “Event Registration.”](#) Beginning with [Section 1.2 “LTP for JavaBeans Overview,”](#) this document primarily discusses LTP.

1.1 Overview of the JavaBeans Component Model

For many years, reusable software components have been desirable in the software engineering field. Reusable components allow for reduced maintenance and development costs as well as increased reliability and rapid application development. The goal of the JavaBeans API is to define a *software component model* so that anyone can create and ship components for the Java Platform.

These components are called JavaBeans components, or equivalently beans, and are defined as classes that conform to the JavaBeans specification. Beans can be reused in many popular development environments.

Note – The term components is overused. Do not confuse JavaBeans components (beans) with Graphical User Interface (GUI) components (descendants of the `java.awt.Component` class). Although all Swing and Abstract Window Toolkit (AWT) components are beans, not all beans are GUI components. Beans can inherit from any class, and they do not need any GUI functionality.

1.1.1 Services Provided by the JavaBeans Component Model

The JavaBeans component model provides these major types of services for the Java Platform:

- **Bean Interface Exposure and Discovery:** This is the mechanism by which beans expose their programmatic interfaces so that they can be driven dynamically by method calls and event notification by JavaBeans-aware applications, such as tools. This mechanism is called introspection and is based on reflection.
- **Bean Properties:** Properties are the public attributes of a JavaBeans component that directly reflect or affect the current state of that bean. Properties are also based on reflection.
- **Event Handling:** This is the mechanism by which beans send notification of events to objects that have requested notification. Notified objects will typically perform some function. This event-handling mechanism is the basis of the publish and subscribe event model used by AWT components since version 1.1 and by all Swing components.
- **Application Builder Support:** The dynamic discovery of interfaces, properties, and behaviors by application builder environments can allow environments to assemble and interactively change the components. An example might be a GUI builder used to interactively create a user interface. Most of these services are provided by the Introspector, which is implemented as the `java.beans.Introspector` class.
- **Bean Packaging:** Since JavaBeans components can be redeployed and redistributed over a network, there is a need to provide a facility to bundle the beans. JavaBeans use the JAR file format to provide the packaging.

- **Persistence:** This is the mechanism for storing the state of beans in a non-volatile place, for example, to a file system or a database. The first persistence mechanism was based on serialization. LTP, added in J2SE, v. 1.4, provides a second format based on XML text. Serialization will be de-emphasized in favor of LTP.

1.1.2 Properties

Properties are discrete, named attributes of a bean that can affect its appearance or its behavior. For example, a GUI button may have a property named “text” that represents the text displayed in the button.

Properties are always accessed through public method calls on their owning object. For readable properties, there is a `getter` method to read the property value; for writable properties, there is a `setter` method to allow the property value to be updated. The properties on a JavaBeans component are inferred by the Introspector based on the bean’s method signatures.

For example, if a bean has a property named `foo` of type `SomeType`, the methods for reading and writing that property value would have the following signatures:

```
public void setFoo(SomeType value)
public SomeType getFoo()
```

The general design pattern for simple properties is:

```
public void set<PropertyName>(<PropertyType> value)
// simple setter
public <PropertyType> get<PropertyName>() // simple getter
```

Note – By the term design patterns, we mean conventional names and type signatures for a set of methods or interfaces that is used for standard purposes. For example, the use of `getFoo` and `setFoo` methods to retrieve and set the values of a `foo` property.

See Section 7 of the JavaBeans specification for more detail about properties and possible notifications sent when the value of a property changes. The JavaBeans specification is available at:

<http://java.sun.com/products/javabeans/docs/spec.html>

1.1.3 Event Registration

Events provide a convenient mechanism for allowing components to be wired together programmatically or in an application builder. The basic mechanism is a “publish and subscribe” model in which the component that is the event source (publisher) notifies all the components interested in processing the information contained within the event (subscribers).

Conceptually, events are a mechanism for propagating state change notifications between a source object and one or more target listener objects. Events have many different uses, but a common example is in user interface toolkits for delivering notifications of mouse actions, widget updates, keyboard actions, and so on.

For event listeners to register themselves with appropriate event source instances, the sources must provide methods for registering, unregistering, and retrieving event listeners. These registration methods conform to a particular set of design patterns that aids in documentation and also enables programmatic introspection.

For example, if a class is a source of `FooEvents`, then it should provide a way for classes interested in receiving those events to register themselves as `FooListeners` on the event source class. The method signatures for registering listeners on the event source would look like:

```
public void addFooListener(FooListener listener)
public void removeFooListener(FooListener listener)
public void FooListener[] getFooListeners()
```

The standard design pattern for event listener registration is:

```
public void add<ListenerType>(<ListenerType> listener)
public void remove<ListenerType>(<ListenerType> listener)
public void <ListenerType>[] get<ListenerType>s()
```

Note – The last method pattern, `get<ListenerType>s()`, was introduced in J2SE, v. 1.4.

1.1.4 Persistent Storage

Making a bean persistent means that it should store parts of its internal state so that it can be resurrected later with a similar appearance and similar behavior. In other words, making a bean persistent means taking a snapshot of the bean where it can be restored between invocations of the application or Java™ Virtual Machine (JVM™).

Before version 1.4, the only storage mechanism was provided by the Java object serialization mechanism. Serialization's most popular use is with Java™ Remote Method Invocation (RMI).

Normally, a bean stores the persistent state for all its public properties. With serialization, the internal state of the bean may also be stored. The keyword `transient` is used to control the level of persistence of the private state.

See Section 5 of the JavaBeans specification for a discussion regarding serialization-based JavaBeans persistence. The JavaBeans specification is available at:

<http://java.sun.com/products/javabeans/docs/spec.html>

1.1.5 Introspection

Introspection is the ability to figure out which properties, events, and methods a bean supports at runtime. Introspection uses low-level reflection to study the method signatures on the target bean and then applies simple design patterns to deduce from those methods what properties, events, and public methods are supported. All this gleaned information is used to create a `BeanInfo` class of meta-information. The `BeanInfo` class provides high-level information to development environments so that they can understand how to interact with it.

The developer of the bean can provide an explicit `BeanInfo` class of meta-information about that bean. If this is the case, the `BeanInfo` class is usually provided for design-time information for the benefit of development environments. If a `BeanInfo` class is not provided, then `java.beans.Introspector` will usually construct a `BeanInfo` class based on the design patterns of the method signatures.

The Introspector provides the following `BeanInfo` meta-information:

- **BeanInfo:** The product of introspection. A class that implements `BeanInfo` provides access to the type of icon that should represent the bean and to the various `FeatureDescriptors`.
- **BeanDescriptor:** A `FeatureDescriptor` that provides global information about the bean, including the human readable description.
- **MethodDescriptor:** A `FeatureDescriptor` that provides information about an individual public method.
- **EventSetDescriptor:** A `FeatureDescriptor` that provides information about the set of methods which make up an event set. It represents the event listeners that can be registered on the bean.
- **PropertyDescriptor:** A `FeatureDescriptor` that provides information about the methods and descriptions of an individual property.

See Section 8 of the JavaBeans specification for a discussion regarding introspection and the types of design patterns that are recognized by the Introspector. The JavaBeans specification is available at:

<http://java.sun.com/products/javabeans/docs/spec.html>

1.1.6 An Example of a Simple Bean

The following code represents an example of a well-written bean that conforms to the basic JavaBeans specification. The comments further elaborate on the design patterns of the methods.

```
import java.beans.*;
import java.util.*;
import java.io.Serializable;

/**
 * This is an example of a simple bean which conforms to
 * the JavaBeans specification. Some of the more notable features
 * of the bean are:
 *
 * + No-argument constructor: This is a requirement so that it
 *   can be correctly instantiated for the Long Term Persistence
 *   mechanism.
 *
 * + Properties: This bean has a single property called "name".
 *   It is accessible with getter
 *   and setter methods. This property is further characterized
 *   as "bound" since it will fire a PropertyChangeEvent when its
 *   value changes.
 *
 * + Event listener registration: Components which are interested in
 *   receiving notifications of PropertyChangeEvents should register
 *   themselves with this object.
 *
 * + Serializable: This class implements the Serializable
 *   interface. This was an old requirement for JavaBeans
 *   Persistence and will be de-emphasized in later releases
 *   of J2SE in favor of the newer, text-based Long Term Persistence
 *   mechanism.
 */
public class SimpleBean implements Serializable {
    /**
     * The publicly accessible state of the bean.
     */
    private String name;
    /**
```

```

    * The PropertyChangeListeners that are interested
    * in receiving PropertyChangeEvents from this object.
    */
private List listeners;

/**
 * The no-argument constructor. This is one of the
 * essential elements of the class in order for it to
 * instantiate within tools and persist correctly.
 */
public SimpleBean() {
    setName("SimpleBean");
}

/**
 * Setter method for the "name" property.
 *
 * The @beaninfo javadoc tag is currently undocumented but is
 * used internally by a custom doclet to automatically
 * generate the BeanInfo meta classes.
 *
 * @param name the string to be used as the name of the bean
 *
 * @beaninfo
 *     bound: true
 *     preferred: true
 *     attribute: visualUpdate true
 *     description: The button's text.
 */
public void setName(String name) {
    String oldName = this.name;
    this.name = name;
    firePropertyChange("name", oldName, name);
}

/**
 * Getter method for the "name" property.
 *
 * @return the name of the bean
 */
public String getName() {
    return name;
}

/**
 * Registration method for objects that want to be notified
 * of PropertyChangeEvents.
 *
 * @param l PropertyChangeListener interested in receiving
 *     notification of bound property changes
 */

```

```

public void addPropertyChangeListener(PropertyChangeListener l) {
    if (listeners == null) {
        listeners = new Vector();
    }
    listeners.add(l);
}

/**
 * De-registration method for PropertyChangeListeners.
 *
 * @param l PropertyChangeListener interested in receiving
 *         notification of bound property changes
 */
public void removePropertyChangeListener(PropertyChangeListener l) {
    if (listeners == null) {
        return;
    }
    listeners.remove(l);
}

/**
 * RetrIREview method that returns all the PropertyChangeListeners
 * registered on this object.
 *
 * This method is a new requirement of the JavaBeans specification
 * for events and was introduced in J2SE v. 1.4.
 *
 * @return an array of listeners registered on this object
 */
public PropertyChangeListener[] getPropertyChangeListeners() {
    if (listeners == null) {
        return new PropertyChangeListener[] {};
    }
    return (PropertyChangeListener[])listeners.toArray(
        new PropertyChangeListener[listeners.size()]);
}

/**
 * A private helper method that notifies all the registered
 * listeners of the property change.
 *
 * This was adapted from java.beans.PropertyChangeSupport.
 */
private void firePropertyChange(String name, Object oldValue,
                                Object newValue) {
    if (oldValue != null && newValue != null
        && oldValue.equals(newValue)) {
        return;
    }
    PropertyChangeEvent evt = new PropertyChangeEvent(this, name,
        oldValue, newValue);

```

```

        PropertyChangeListener[] targets;
        synchronized (this) {
            targets = getPropertyChangeListeners();
        }
        for (int i = 0; i < targets.length; i++) {
            targets[i].propertyChange(evt);
        }
    }
}

```

1.2 LTP for JavaBeans Overview

LTP for JavaBeans is a new persistence model for JavaBeans components. See [Section 1.2.1 “JavaBeans and LTP”](#) for a brief, practical description of what makes an object a bean. A longer explanation is available in [Section 1.1.6 “An Example of a Simple Bean.”](#) LTP is designed to handle the process of converting a parent/child “tree” or “graph” of beans to and from an XML-based text format.

LTP is used for the long-term storage of the state of a component in terms of its public API. Previously, serialization was advocated as the solution for storage of bean state. There are several problems with using serialization for long-term storage:

- Serialization saves all the private state of the bean. Thus if the implementation of the class has changed slightly, then the bean will not be deserialized correctly.
- When an object fails to deserialize correctly, it throws an exception and returns. No aspect of the serialized object is recovered.
- The serialized object is stored in a binary form. The developer cannot fix an error in the serialized object without having the same version of the JVM and set of class files which originally serialized the object.

LTP for JavaBeans addresses all these issues. The following objectives were used to guide the design process:

- Resilience to changes in the versions of both JVMs and class libraries.
- Fault tolerance, allowing an archive to load even when part of it is damaged.
- Archives exclusively in terms of public APIs, not private implementations.
- Textual (ASCII) output that can be edited with standard tools (XML format).
- Comprehensive use of defaults (redundancy elimination) to minimize file size.
- Performance that varies linearly with the number of nodes in the graph.

LTP for JavaBeans is part of the `java.beans.*` package. You can find articles and tutorials on LTP published or referenced at:

<http://java.sun.com/products/javabeans>

1.2.1 JavaBeans and LTP

LTP is based on the JavaBeans design patterns outlined in the JavaBeans specification document outlined at:

<http://java.sun.com/products/javabeans/docs/spec.html>

Objects must conform to the JavaBeans specification to be automatically archivable with LTP. The two main features that the bean must have are:

- A bean must have a no-argument constructor.
- All public state or properties should be set or retrieved using JavaBeans design patterns. For example, a property `foo` should be set and retrieved using `void setFoo(FooType)` and `FooType getFoo()` methods.

For more information about beans, see:

- Java Developer Connection

<http://developer.java.sun.com/developer/onlineTraining/Beans/>

- The Java Tutorial

<http://java.sun.com/docs/books/tutorial/javabeans/>

1.2.2 Features, Advantages, and Benefits of LTP

The benefits of using LTP are that it addresses the problems of serialization while accomplishing the design goals described in [Section 1.2 “LTP for JavaBeans Overview.”](#)

Important features of LTP:

- Easy to use. The API is very similar to the API used for serialization. Developers who understand and use serialization can easily understand LTP.
- Human- or machine-readable XML format. Archives created can be modified by text editors or parsed, using the new XML packages in J2SE, v. 1.4.

Two classes, `java.beans.XMLEncoder` and `java.beans.XMLDecoder`, write to and read from the XML persistence format. Classes that inherit from `java.beans.PersistenceDelegate` are helper classes to make non-conforming beans archivable.

1.2.3 Features or Services Not Provided With LTP

One product limitation of LTP is that archives of anonymous inner classes (for example, an event adapter that provides an action to a button press on a user interface) are not archivable. This limitation and possible solutions are discussed in detail in [Section 6.1 “Configuration Files.”](#)

1.3 Other Introductory Material

For more information on technologies related to JavaBeans, see:

<http://java.sun.com/products/javabeans/docs/related.html>

- Java Core Reflection API

<http://java.sun.com/j2se/1.4/docs/guide/reflection/>

- Java Archive (JAR) Features

<http://java.sun.com/j2se/1.4/docs/guide/jar/>

1.4 Specialized Terminology

1.4.1 Glossary

Archive: A text file that is the product of encoding an object graph using the LTP mechanism.

Decoded, Decoding: The process of reading an XML-formatted file or stream and reconstituting the encoded object graph.

Encoded, Encoding: The process of taking a JavaBeans object graph and writing it to an external XML-formatted stream or file.

LTP: Unofficial abbreviation for Long Term Persistence for JavaBeans.

Object Graph: A tree of an aggregate JavaBeans component that is comprised of JavaBeans components. The relationship may be expressed in terms of properties, containment, or parent-child relationships, as long as the methods that set and retrieve the aggregate parts conform to JavaBeans design patterns.

Persistence Delegate: A helper class that provides meta-information, enabling objects of another class to be made persistent. A persistence delegate is registered with the encoder and does not need to be present in the decoding process.

Reflection: An API that enables Java code to discover information about the fields, methods, and constructors of loaded classes, and to use reflected fields, methods, and constructors to operate on their underlying counterparts on objects within security restrictions. For more information, see:

<http://java.sun.com/j2se/1.3/docs/guide/reflection/>

Serialization: The encoding of objects and the objects reachable from them into a stream of bytes. The serialization API also supports the complementary reconstruction of the object graph from the stream. Serialization is used for lightweight persistence and for communication through sockets or RMI. For more information, see:

<http://java.sun.com/j2se/1.4/docs/guide/serialization/>

2 Product Changes for Version 1.4

LTP is a new API in J2SE, v. 1.4. It was developed through the Java Community Process as JSR-057 at:

<http://jcp.org/jsr/detail/057.jsp>

Before the version 1.4 Beta releases, two early versions of LTP were included in Swing Connection articles. Both were introduced as demos or proofs of concept and did not contain an associated version numbering scheme.

2.1 Previous Versions of LTP for JavaBeans

The first version was in Beta 1. In Beta 2, LTP was changed to contain support for terse representation of static constants and array elements, as described in the next section. The Beta 3 release contains bug fixes but no specification changes.

J2SE, v. 1.4, Beta 2, has two additions to the XML schema to make it more clear.

2.1.1 Static Constants

Starting with version 1.4, Beta 2, the values of static constants may be written using the `class` and `field` attributes to specify the declaring class and field name of the constant, respectively. This was added to make the output more terse.

As an example, the following XML fragment:

```
<void class="javax.swing.JTable" method="getField"
  <stringAUTO_RESIZE_OFF</string
  <void id="Integer0" method="get"
    <null/
  </void
```

```
</void  
<object idref="Integer0"/
```

is shortened to:

```
<object class="javax.swing.JTable" field="AUTO_RESIZE_OFF"/
```

2.1.2 Array Elements

The `<array>` tag can be used to represent arrays with the `class` and `length` attributes denoting the subtype and the length of the array. In version 1.4, the `length` attribute can be omitted, and the values may be specified as if each array has a constructor that takes all the elements as arguments.

2.2 Backward and Forward Compatibility With Other Versions

Archives created with 1.4 Beta versions should unarchive correctly in J2SE, v. 1.4.

3 Using and Supporting LTP for JavaBeans

3.1 User's Guides and Tutorials

A series of articles on The Swing Connection introduce the LTP framework:

- Introduction and goals
<http://java.sun.com/products/jfc/tsc/articles/persistence>
- JavaOne 2000 update including information about builder companies
<http://java.sun.com/products/jfc/tsc/articles/persistence2>
- Discussion of the XML schema
<http://java.sun.com/products/jfc/tsc/articles/persistence3>
- For more advanced XMLEncoder usage, such as custom persistence delegates, the owner property
<http://java.sun.com/products/jfc/tsc/articles/persistence4>

The JavaBeans trail of The Java Tutorial will be updated to incorporate LTP:

<http://java.sun.com/docs/books/tutorial/javabeans/>

3.2 Using LTP for JavaBeans in a Development Environment

We have been working with tool vendors, such as JBuilder, Visual Age for Java, Visual Cafe, and SAS, to get their support for incorporating LTP as a transfer / export file format for persistent designs. Ideally, in a couple of years, object graphs will be persistent in an XML format by selecting the export function in an Integrated Development Environment (IDE).

3.3 Tools and Utilities

The Bean Builder is a tool that uses LTP. For more information, see:

<http://java.sun.com/products/jfc/tsc/articles/persistence/#downloadbuilder>

This builder includes source code and is a demonstration of how to use the APIs to persist and reconstitute designs. This builder can also be used as a test container to import XML-encoded object graphs and examine them.

3.4 Localization and Internationalization

Internationalization strategies are discussed at:

<http://java.sun.com/products/jfc/tsc/articles/persistence4>

A more detailed discussion can be found in bug 4446154. See:

<http://developer.java.sun.com/developer/bugParade/bugs/4446154.html>

4 Code Samples

The following section may contain actual software programs in source code form. This source code is made available for developers to use as needed, pursuant to the terms and conditions of this license located at:

<http://access1.sun.com/SRDs/BSD.html>

4.1 Writing an Object Graph to an OutputStream

The most common use of LTP is to write an object graph to an output stream. The stream can represent a file in a file system or any `OutputStream`, such as the standard `PrintStreams`, `System.out`, or `System.err`.

The following utility method will write an object graph to the file system with the specified filename.

```
/**
 * Writes the root object to an archive.
 *
 * @param root      the root object of an object graph
 * @param filename  the name of the file to save the object graph to
 */
public static void writeArchive(Object root, String filename) {
    try {
        BufferedOutputStream out =
            new BufferedOutputStream(new FileOutputStream(filename));
        XMLEncoder encoder = new XMLEncoder(out);
        encoder.writeObject(root);
        encoder.close();
    } catch (Exception ex) {
        ex.printStackTrace();
    }
}
```

The following utility method is a general-purpose write method that encodes the object tree to an `OutputStream`. It installs an exception listener if one is passed as an argument.

```
/**
 * Writes the root object out to the stream.
 *
 * @param root the root object of an object graph
 * @param filename the name of the file to save the object graph to
 * @param l exception listener to handle caught exceptions
 */
public static void writeArchive(Object root,
                                OutputStream stream,
                                ExceptionListener l) {
    XMLEncoder encoder = new XMLEncoder(stream);
    if (l != null) {
        encoder.setExceptionListener(l);
    }
    encoder.writeObject(root);
    encoder.close();
}
```

The following snippet of code is a good way to test the archivability of object graphs. These three lines do not have to be wrapped in a try/catch block and will print to the console that started the JVM.

```
XMLEncoder e = new XMLEncoder(System.out);
e.writeObject(rootObject);
e.close();
```

4.2 Reading an Object Graph from an InputStream

Reading an object graph from an `InputStream` is the second most common use of LTP.

The following utility method reads an archive from the file system.

```
/**
 * Reads the named archive and returns the root object.
 *
 * @param filename an XML archive that was created with LTP
 * @return the decoded object graph or null if an error occurred
 */
public static Object readArchive(String filename) {
    FileInputStream in = null;
    XMLDecoder decoder = null;
    Object root = null;
```



```

    try {
        in = new FileInputStream(filename);
        decoder = new XMLDecoder(in);
        root = decoder.readObject();
        decoder.close();
    } catch (Exception ex) {
        System.out.println(ex.getMessage());
        return null;
    }
    return root;
}

```

The following utility method is a general-purpose read method that reads an archive from an `InputStream`. It installs an exception listener if one is passed in.

```

/**
 * Reads the named archive and returns the root object.
 *
 * @param input an InputStream
 * @param l ExceptionListener
 * @return the decoded object graph or null if an error occurred
 */
public static Object readArchive(InputStream input, ExceptionListener l) {
    XMLDecoder decoder = new XMLDecoder(input);
    if (l != null) {
        decoder.setExceptionListener(l);
    }
    Object root = decoder.readObject();
    decoder.close();
    return root;
}

```

4.3 Preventing Some Properties From Archiving

A situation could arise in which the developer does not want some properties to be written to the archive. LTP provides a mechanism for marking properties as “transient” so that the encoder will skip them. This is analogous to using the `transient` keyword in the context of serialization except that the decision to make properties transient can be done in user code rather than in the library.

To prevent a property from archiving, set the “transient” attribute to `true` on the `PropertyDescriptor` that corresponds to the property. A `PropertyDescriptor` contains information about a particular bean property, such as the bean’s getter and setter methods. It also contains an attribute table that LTP uses to store and retrieve information about how that item will persist.

The property descriptors for a bean are retrieved from the `BeanInfo` object that represents the bean. `BeanInfo` objects are returned by the `Introspector` class. For example, the following code gets the `BeanInfo` for the `JButton` class: `Introspector.getBeanInfo(JButton.class)`.

The following utility method retrieves the property descriptor corresponding to a particular property of a bean.

```
/**
 * Returns the descriptor for a property defined by a particular
 * class.
 *
 * @param type the class that defines the property
 * @param propertyName the name of the property;
 *                    for example, "text" for getText/setText
 * @return the PropertyDescriptor or null
 */
public static PropertyDescriptor getPropertyDescriptor(Class type,
                                                    String propertyName) {
    BeanInfo info = null;
    try {
        info = Introspector.getBeanInfo(type);
    } catch (IntrospectionException ex) {
        ex.printStackTrace();
        return null;
    }
    PropertyDescriptor[] propertyDescriptors =
        info.getPropertyDescriptors();
    for (int i = 0; i < propertyDescriptors.length; i++) {
        PropertyDescriptor pd = propertyDescriptors[i];
        if (propertyName.equals(pd.getName())) {
            return pd;
        }
    }
    return null;
}
```

Let's use this method to make the model property of `JButton` transient. The first step is to retrieve the property descriptor that corresponds to the model property from the `JButton`'s `BeanInfo`. Once we have that, we mark the transient property `true`. In the following example, we make the model property transient after archiving `button1`, so the model property will be archived for `button1` but not for `button2`. Because `JToggleButton` does not inherit from `JButton`, even though it inherits the model property from `AbstractButton` just as `JButton` does, the toggle button's model property will be archived.

```
// NOTE: This code uses the utility methods shown previously,
//       which we've put in a class named ArchiveUtilities.
// The "model" property for this button will be archived.
```

```

JButton button1 = new JButton();
button1.setText("Button 1");
ArchiveUtilities.writeArchive(button1, "button1.xml");
// Make the "model" property of transient for JButtons.
PropertyDescriptor pd =
    ArchiveUtilities.getPropertyDescriptor(JButton.class, "model");
pd.setValue("transient", Boolean.TRUE);
// The "model" property for this button will NOT be archived.
JButton button2 = new JButton();
button2.setText("Button 2");
ArchiveUtilities.writeArchive(button2, "button2.xml");
// The "model" property for this button WILL be archived.
JToggleButton tb = new JToggleButton();
tb.setText("Toggle Button");

```

Note – This will work correctly only on J2SE, v. 1.4, Beta 3 or later.

4.4 Using a ByteArrayOutputStream

XMLEncoder can write to any type of an OutputStream. The developer may want to write to a `java.io.ByteArrayOutputStream` if they wish to defer writing to a file system or send the object tree over the network. This technique of reading and writing from a `ByteArrayOutputStream` can also be used to test the archivability of classes and object trees without having to write to the file system.

The following code uses the object stream versions of the utility methods shown previously for `writeArchive` and `readArchive` to demonstrate writing to and reading from a byte array stream. An object tree is constructed, archived to a `ByteArrayOutputStream`, and restored.

```

DefaultMutableTreeNode root = new DefaultMutableTreeNode("root");
for (int i = 0; i < 10; i++) {
    root.add(new DefaultMutableTreeNode("child"));
}
ByteArrayOutputStream os = new ByteArrayOutputStream();
ArchiveUtilities.writeArchive(root, os, exListener);

ByteArrayInputStream in = new
ByteArrayInputStream(os.toByteArray());
DefaultMutableTreeNode newRoot =
    (DefaultMutableTreeNode)ArchiveUtilities.readArchive(
        in, exListener);

```

5 Troubleshooting

Most of the diagnostic messages associated with LTP failures are printed to the console that started the JVM. LTP skips over failures and exceptions and attempts to archive the next element in the object graph.

5.1 Missing No-Argument Constructor

An error message, such as the following, indicates that the class to be archived lacks a no-argument constructor:

```
java.lang.InstantiationException: basic.NoNullCtor
Continuing ...
java.lang.Exception: discarding statement
XMLEncoder0.writeObject(NoNullCtor0);
Continuing ...
```

or

```
java.lang.InstantiationException: java.awt.image.BufferedImage
Continuing ...
java.lang.RuntimeException: failed to evaluate:
<unbound>=BufferedImage.new();
Continuing ...
```

The solution, assuming the bean cannot be changed, is to provide a `PersistenceDelegate` for the class that creates constructor arguments appropriate for specific instances. The context is written in terms of the publicly accessible properties.

Often the persistence delegate can be created using the `String[]` constructor of `DefaultPersistenceDelegate`. This constructor specifies the arguments for the bean's constructor, assuming that the arguments are properties of the bean. The persistence delegate should be added to the encoder before the `writeObject` method is called.

The following is an example of a class without a no-argument constructor:

```
public class NoNullCtor {
    private int foo, bar;
    public NoNullCtor(int foo, int bar) {
        this.foo = foo;
        this.bar = bar;
    }
    public void setFoo(int foo) {
        this.foo = foo;
    }
    public int getFoo() {
        return foo;
    }
    public void setBar(int bar) {
        this.bar = bar;
    }
    public int getBar() {
        return bar;
    }
}
```

The two arguments `foo` and `bar` correspond to simple properties for which well-formed getters and setters are available. To make this class persist correctly, the constructor for the `DefaultPersistenceDelegate` should provide context to the two-argument constructor of the `NoNullCtor` class:

```
XMLEncoder e = new XMLEncoder(System.out);
e.setPersistenceDelegate(NoNullCtor.class,
    new DefaultPersistenceDelegate(new String[] { "foo", "bar" }));
e.writeObject(new NoNullCtor(10, 20));
e.close();
```

For more examples of persistence delegates and how to control the encoding of a tree of objects, please see:

<http://java.sun.com/products/jfc/tsc/articles/persistence4>

5.2 Product Limitations

5.2.1 Anonymous Inner Classes are not Archivable

The most significant limitation of LTP is that it cannot save anonymous inner classes. These classes are usually added as anonymous listeners, as glue code, or as event adapter classes that hook up user interface elements.

For example, the following code snippet uses an anonymous inner class, so it will not archive correctly and the listener will be rejected:

```
 JButton button = new JButton("My Button");
 button.addActionListener(new ActionListener() {
     public void actionPerformed(ActionEvent evt) {
         System.out.println("Button Pressed");
     }
 });
```

There are a number of workarounds if the developer has access to the source. If the developer does not have access to the source code then nothing can be done to archive this code snippet correctly. Fortunately, a partial archive without the listener will be created.

5.2.1.1 First Workaround: Make the Listener a Bean

One simple workaround is to make the anonymous inner class a first-class bean. As long as the class is reachable from the class path, it will be archived and can be reconstituted. For example, the preceding inner class could be re-implemented as a bean like this:

```
public class ActionBean implements ActionListener {
    // The default constructor created by the compiler takes
    // care of the no-argument constructor requirement.
    // This bean has no properties, and thus no need for
    // setter/getter methods.
    public void actionPerformed(ActionEvent evt) {
        System.out.println("Button Pressed");
    }
}
```

The code to add the listener must be adapted to add the listener bean instead:

```
button.addActionListener(new ActionBean());
```

5.2.1.2 Second Workaround: Write a Dynamic Proxy

A new class, `java.beans.EventHandler`, was added to J2SE, v. 1.4, to support the creation of dynamic proxy adapters. Anonymous inner classes that rely on properties can be rewritten as dynamic proxy adapters. For example, consider this anonymous inner class:

```

JButton button = new JButton("My Button");
JLabel label = new JLabel("My Label");
button.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent evt) {
        label.setText(((JButton)e.source()).getText());
    }
});

```

It can be rewritten in terms of dynamic proxy adapters as follows:

```

JButton button = new JButton("My Button");
JLabel label = new JLabel("My Label");
button.addActionListener((ActionListener)EventHandler.create(
    ActionListener.class, label, "text", "source.text"));

```

For details, see the API documentation at:

<http://java.sun.com/j2se/1.4/docs/api/java/beans/EventHandler.html>

Also, there is a Swing Connection article about the dynamic proxy adapters at:

<http://java.sun.com/products/jfc/tsc/articles/generic-listener2/>

5.3 Common User Questions

A list of frequently asked questions for JavaBeans is maintained at:

<http://java.sun.com/products/javabeans/FAQ.html>

5.4 Troubleshooting Utilities

When an error occurs in the process of persisting or restoring an object graph, the error is skipped over, a brief error message may be written to the `System.err` stream, and the rest of the graph or archive is processed.

To elaborate on the error message, you may add an exception handler that implements `java.beans.ExceptionListener` to either `XMLEncoder` or `XMLDecoder`. The exception handler is notified when the input or output stream catches recoverable exceptions. You may implement the `exceptionThrown` method to print a detailed stack trace or append information to a log file.

For example, the following implementation of an `ExceptionListener` will print the stack trace from the caught exception to the `System.err` stream:

```
XMLEncoder encoder = new XMLEncoder(System.out);
encoder.setExceptionListener(new ExceptionListener() {
    public void exceptionThrown(Exception e) {
        System.err.println(
            "Exception thrown during serialization:");
        e.printStackTrace();
    }
});
```

5.5 Common Developer Problems

5.5.1 Why Fields That Are Marked With the Transient Keyword Still Persist

The LTP mechanism is completely independent from the serialization mechanism. Serialization will write out the state of the private fields unless explicitly marked as transient.

The LTP uses the public API, so if the private field is exposed by public getters and setters, then it will be written. In order to prevent the state of this property from being written, you must mark the corresponding `PropertyDescriptor` with the transient keyword.

See the Swing Connection article *Using XMLEncoder* at:

<http://java.sun.com/products/jfc/tsc/articles/persistence4/index.html>

Also, see the section on *Making Properties Transient* at:

<http://java.sun.com/products/jfc/tsc/articles/persistence4/index.html#transient>

5.6 Known Bugs and Their Workarounds

The category and subcategories for JavaBeans are: `java`, `classes_beans`.

Please visit `bugParade` located at:

<http://developer.java.sun.com/developer/bugParade>

5.6.1 Bug 4646747: Custom persistence delegate lost after memory stress

Problem: If you use a custom persistence delegate by invoking `setPersistenceDelete` on `XMLEncoder` and then the JVM runs low on memory, `XMLEncoder` loses its reference to the persistence delegate. Thereafter, the custom persistence delegate will not be used.

Workaround: Maintain a strong reference to all `BeanInfos` corresponding to classes that have custom persistence delegates.

The complete code for the workaround is shown in the bug report, which you can view here:

<http://developer.java.sun.com/developer/bugParade/bugs/4646747.html>

6 Key Files and Directories

6.1 Configuration Files

No special configuration files are required for LTP.

Since LTP uses the JavaBeans Introspector for most of its processing, the existence of explicit `BeanInfo` classes may have an effect on the created archives. For example, an explicit `BeanInfo` may choose to hide public properties. If an archive is created with an explicit `BeanInfo`, then that explicit `BeanInfo` may be required to reconstitute the object graph from the input stream.

For example, the file `$JAVA_HOME/lib/dt.jar` contains the `BeanInfo` classes for all the JFC (Swing) components. This file is in the same location for Microsoft Windows. If this file is included in the class path, the meta-information will be used in the archiving process, but it should not have an affect on the archives.

7 Installing and Configuring LTP for JavaBeans

JavaBeans is a component of the J2SE Software Developers Kit (J2SDK). It is installed as part of the J2SDK.

For information on:

- Downloading, installing, and uninstalling the J2SDK
- Requirements and dependencies of the J2SDK
- Product distribution and licensing of the J2SDK

Please see the J2SE Overview SRD available at:

http://access1.sun.com/SRDs/access1_srds.html

8 Reference Information

8.1 Product Information

LTP for JavaBeans information, articles, and tutorials are indexed at the JavaBeans site:

<http://java.sun.com/products/javabeans/>

8.2 Technical Documentation

Most of the documentation for the LTP for JavaBeans API is incorporated in the API documentation:

<http://java.sun.com/j2se/1.4/docs/api/java/beans/package-summary.html>

You can download the specification from:

<http://java.sun.com/products/javabeans/docs/spec.html>

Or see it in .pdf from:

<http://java.sun.com/products/javabeans/docs/beans.101.pdf>

8.3 Frequently Asked Questions

A list of frequently asked questions for JavaBeans is maintained at:

<http://java.sun.com/products/javabeans/FAQ.html>

A JDC Forum for JavaBeans is at:

<http://forum.java.sun.com/forum.jsp?forum=39>

This forum is for the JavaBeans development community and is generally not monitored by members of the JavaBeans team.

8.4 Tutorials and Other References

A series of articles on The Swing Connection introduce the LTP framework:

- Introduction and goals
<http://java.sun.com/products/jfc/tsc/articles/persistence>
- JavaOne 2000 update including information about builder companies
<http://java.sun.com/products/jfc/tsc/articles/persistence2>
- Discussion of the XML schema
<http://java.sun.com/products/jfc/tsc/articles/persistence3>
- For more advanced XMLEncoder usage, such as custom persistence delegates, the owner property
<http://java.sun.com/products/jfc/tsc/articles/persistence4>

The JavaBeans trail of The Java Tutorial will be updated to incorporate LTP:

<http://java.sun.com/docs/books/tutorial/javabeans/>